# AM9511 Interface for the 6502

Carl Harris

December 2024

The AMD AM9511 Arithmetic Processing Unit, first produced in the late 1970s, provides high performance fixed or floating point arithmetic as well as common transcendental functions. It provides an excellent enhancement to any retro-computer that will be used to perform fixed-point or floating-point mathematical operations. At present, these parts are obtainable from various suppliers as new-old stock or used stock.

This article describes an interface allowing the AM9511A to be directly connected to the bus of the 6502 microprocessor. It is adapted from the work of Kissel or Currie[1] in which they developed an experiment using a MOS 6502 microprocessor to control four AM9511A floating-point arithmetic units. It also includes elements of Hart's MICROCRUNCH[2] which used the AM9511A (Intel 8231A) interfaced with a (Rockwell) 6502.

## 1 Bus Interface Design

The AM9511 uses an asynchronous interface design – its timing is not derived from the clock of the host microprocessor. This allows the AM9511A to be clocked at 4 MHz while, in the case of the original 6502, the system clock is only 1 MHz. A disadvantage of this approach is that the interface has some strict timing requirements to ensure that the actions of the AM9511 or the microprocessor are effectively coordinated, despite the lack of a shared clock.

The bus signal timing requirements of the AM9511 are generally easy to satisfy using microprocessors such as the 8080, 8085, or Z80, all of which are designed to use multiple clock cycles for each machine cycle. In such systems, the bus control signals for addressing or read/write control are changed over the course of multiple clock cycles, with addressing (chip selection) happening first, followed by changes to read or write signals in later cycles. At the end of a cycle, the read and write signals change first, before changes to the address lines.

In the 6502, after the falling edge of the clock, addressing and read/write selection are stabilized at effectively the same time. At the next rising clock edge, a read or write operation occurs, ending at the subsequent falling edge at which time the addressing and read/write selection signals are subject to change again.

The AM9511 requires the address input ($C/\overline{D}$) to be stable before the chip select ($\overline{CS}$) and read ($\overline{RD}$) or write ($\overline{WR}$) signals are asserted. The minimum delay between the fall of $\overline{CS}$ and the fall of either $\overline{RD}$ or $\overline{WR}$ is given as zero, which simply means that $\overline{RD}$ or $\overline{WR}$ cannot precede $\overline{CS}$. This is generally easy to satisfy in a 6502-based system.

The first complication involves write operations. The AM9511 requires a minimum positive delay between the rise of the $\overline{WR}$ signal (at he end of a write operation) and the rise of the $\overline{CS}$

[1] Kissel, R and Currie, J: *Hardware Math for the 6502 Microprocessor*; July 1985; NASA TM-86517

[2] Hart, J: *MICROCRUNCH: An Ultra-Fast Arithmetic Computing System (part 1)*; August 1981; MICRO - the 6502/6809 Journal

signal. Depending on the specific AM9511 part, this delay requirement could be as high as 60 ns. Obviously, this requirement precludes direct use of the $R/\overline{W}$ signal as the input to $\overline{WR}$, since $R/\overline{W}$ changes state coincident with the address lines soon after the falling edge of the $PHI2$ clock.

As suggested by Hart, the solution here uses a monostable multivibrator to effectively control the length of the low pulse on the $\overline{WR}$ signal. As shown in Figure 1, the circuit uses a 74HC123 monostable multivibrator. This part is used because it is easy to get a fixed delay in the monostable pulse, without regard to the length of the trigger pulse. The combination of a 68 pF capacitor and a 2.4 KΩ resistor provides an output pulse of approximately 75 ns which is sufficient to meet the delay requirement for any AM9511 part, while still being significantly shorter than one half of the $PHI2$ clock period.

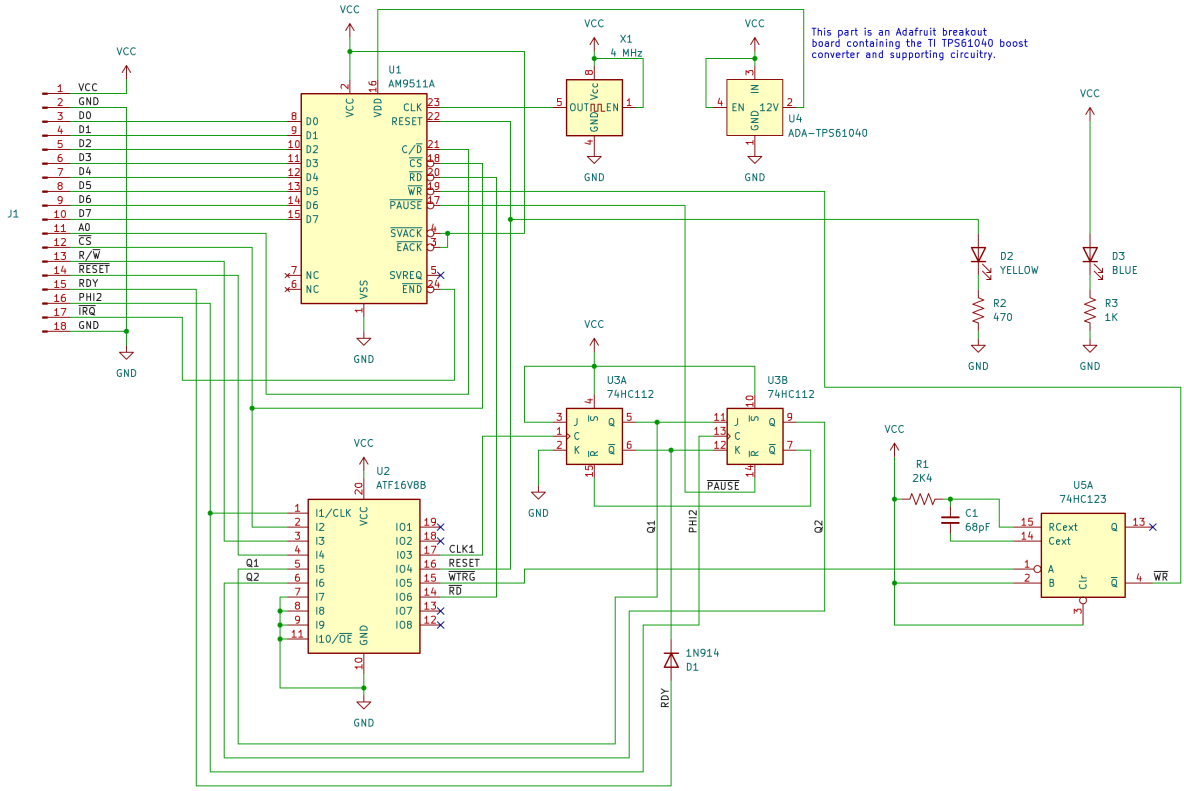


Figure 1: Circuit Schematic

In a read operation, it is assumed that the microprocessor will set $C/\overline{D}$, then assert $\overline{CS}$ and $\overline{RD}$. The AM9511 then asserts its $\overline{PAUSE}$ output, which remains low until the requested data is available on lines $D[0..7]$. When the data is available, the AM9511 releases $\overline{PAUSE}$ and lines $D[0..7]$ remain stable until $\overline{RD}$, $\overline{CS}$, or $C/\overline{D}$ changes. The minimum delay between the rise of $\overline{RD}$ and the rise of $\overline{CS}$ is given as zero – in particular, this implies that $\overline{RD}$ cannot rise *after* $\overline{CS}$.

The intent of the $\overline{PAUSE}$ signal in a read operation is to force the microprocessor to wait until data requested from the AM9511 becomes available on the bus. The 6502 has an active high $RDY$ input that inserts wait states for this purpose. A complication arises from the fact that the 6502 observes the state of this input on the rising edge of the $PHI2$ clock - i.e. at the instant at which the 6502 would perform the read operation, if $RDY$ is low it will instead wait until the next rising edge of the clock at which time it will again observe the state of RDY. After $\overline{RD}$ is asserted, the

AM9511 may require as much as 150 ns before $\overline{PAUSE}$ is asserted. Depending on the $PHI2$ clock frequency and the time that it takes for the address and $R/\overline{W}$ lines to settle, this could occur *after* the rising edge of the clock cycle in which the read operation was initiated.

Asserting the AM9511's $\overline{RD}$ after the falling edge of the clock is further complicated by the fact that the 6502 can arbitrarily change the state of the address lines and $R/\overline{W}$ immediately after the falling edge of the $PHI2$ clock. We must be careful to avoid asserting $\overline{RD}$ spuriously while the 6502's signals are settling – doing so could inadvertently trigger a read operation, unexpectedly disrupting the state of the AM9511's stack.

To solve both of these complications with read operations, the solution here is basically the same as that used by Kissel and Currie – a pair of JK flip-flops that latch $RDY$ in the low state at the beginning of an operation, and reset it to the high state after the $\overline{PAUSE}$ signal rises. The state of these flip-flops is also used as an enable for the $\overline{RD}$ signal.

This circuit uses a programmable logic device (ATF16V8) to implement the necessary combinatorial logic. This could also be done with discrete 7400 series logic circuits, but the ATF16V8 is readily available and easily programmed using open-source tools Galette[3] and Minipro[4]. Figure 2 shows the logic programmed into the PLD.

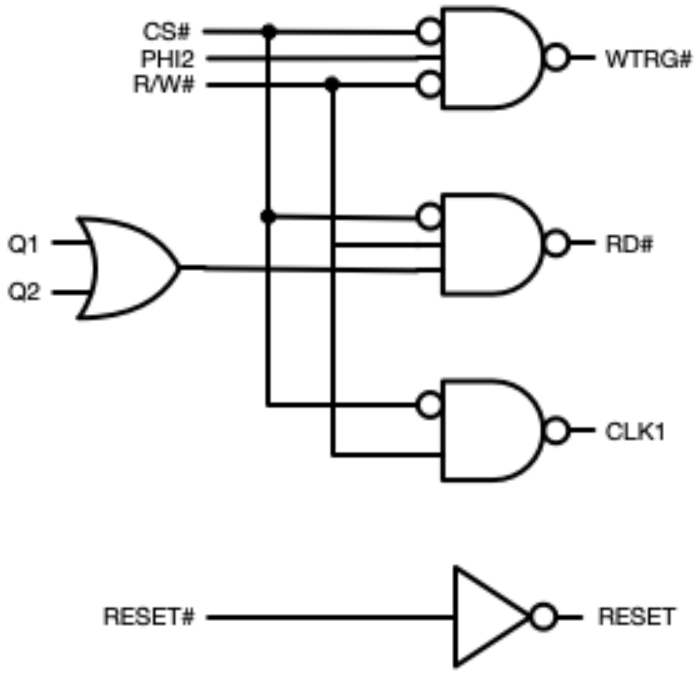


Figure 2: Combinatorial Logic

With this logic, the $\overline{WTRG}$ signal is asserted at the rising edge of $PHI2$ if the $\overline{CS}$ signal is asserted and $R/\overline{W}$ is low. This output provides the trigger input for the 74HC123 used to produce the $\overline{WR}$ pulse for the AM9511.

[3] *Galette: A GAL assembler, largely galasm-compatible and written in Rust*; `https://github.com/simon-frankau/galette`

[4] *Minipro: An open source program for controlling the MiniPRO TL866xx series of chip programmers*; `https://gitlab.com/DavidGriffith/minipro`

The $Q1$ and $Q2$ inputs to this logic are the corresponding outputs of the JK flip-flops. These are logically OR'ed as an enable signal to prevent $\overline{RD}$ from being asserted spuriously. The CLK1 output connects to the clock input of the first flip-flop. Since the 74HC112 has a negative edge trigger, the falling edge of this signal (when $\overline{CS}$ is asserted and R/$\overline{W}$ is high) clocks a logic one into the first flip-flop, and simultaneously drives the $\overline{RD}$ signal low.

The $\overline{Q1}$ output of the first flip-flop is connected to the 6502's $RDY$ input, so a logic one in this flip-flop signals to the 6502 that it should wait at the next rising edge of $PHI2$. The clock input of the second flip-flop is tied to $PHI2$, so at the next falling edge of $PHI2$ for which the AM9511's $\overline{PAUSE}$ output is not asserted, the second flip-flop will clock in a logic one. Output $Q2$ will be high, so $\overline{RD}$ will continue to be asserted even though output $\overline{Q2}$ will reset the first flip-flop, signaling to the 6502 that it can continue. The next rising edge of the $PHI2$ will complete the read operation, and the subsequent falling edge of $PHI2$ will cause the second flip-flop to clock in a logic zero (because $J2$ is low while $K2$ is high).

The only other significant elements of the circuit are the 4 MHz clock oscillator and a boost converter.

The AM9511A-4DC can run at 4 MHz. Other parts, such as the AM9511A-1DC require a clock speed no greater than 3 MHz. I decided to use a dedicated on-board clock oscillator mostly as a matter of convenience. Of course, the AM9511 could be clocked by the system clock (with an appropriate divider circuit if necessary).

The AM9511 requires a +12V DC bias input. I choose to use a boost converter since the current requirement is quite small, and doing so avoids the complication of a having multiple DC supplies at different voltages. The boost converter depicted in the schematic is a breakout board made by Adafruit. However, any +5 VDC to +12 VDC boost converter should be fine – only about 100 mA current is needed at +12 VDC.

## 2 Other Considerations

The interface described herein does not provide a mechanism to wait after a write operation. This means that even though the AM9511 asserts the $\overline{PAUSE}$ signal while executing a requested operation, the 6502 will not wait until the requested operation is completed. This allows the programmer to decide how best to use the time during which an operation is executing. One simple approach is to simply poll the BUSY flag in the AM9511's status register in a loop to determine when a requested operation has completed execution.

Alternatively, the hardware could make use of the $\overline{END}$ signal (or $SVREQ$ signal) as an interrupt source to allow other work to be completed concurrently while the AM9511 is executing operations. The module described here can connect $\overline{END}$ to the 6502's $\overline{IRQ}$ input if desired. The $\overline{END}$ signal is an open-drain output, allowing it to be easily incorporated into a design that uses a wired-OR configuration for multiple devices connected to the 6502's $\overline{IRQ}$ input.

Furht and Lee[5] describe an interrupt-driven approach for the 8085 microprocessor that could be adapted for the 6502. Using such a design, is it possible to enqueue a sequence of interdependent operations in main memory, pushing operands and operations onto the AM9511's stack as soon as it is able to accommodate them.

[5]Furht, B and Lee, P: *An Efficient Software Driver for AM9511 Arithmetic Processor Implementation*; June 1984; IEEE Micro, Volume 4 Issue 3

# 3 Observed Timing

The following oscilloscope plots show key aspects of the interface timing as observed using a WDC 65C02 clocked at 1.8432 MHz and the AM9511A clocked at 4 MHz.

## 3.1 Write Operation

Figure 3 shows the timing for a write operation. A write operation targeting the dataregister versus the command register differs only in the value of the $C/\overline{D}$ signal (shown here as $A0$) – it is high here signifying a write to the command register. Note that the $\overline{WR}$ pulse falls soon after the rising edge of $PHI2$ but rises approximately 75 ns before either $\overline{CS}$ or $A0$ changes state. This satisfies what amounts to the most critical timing constraint of the AM9511.

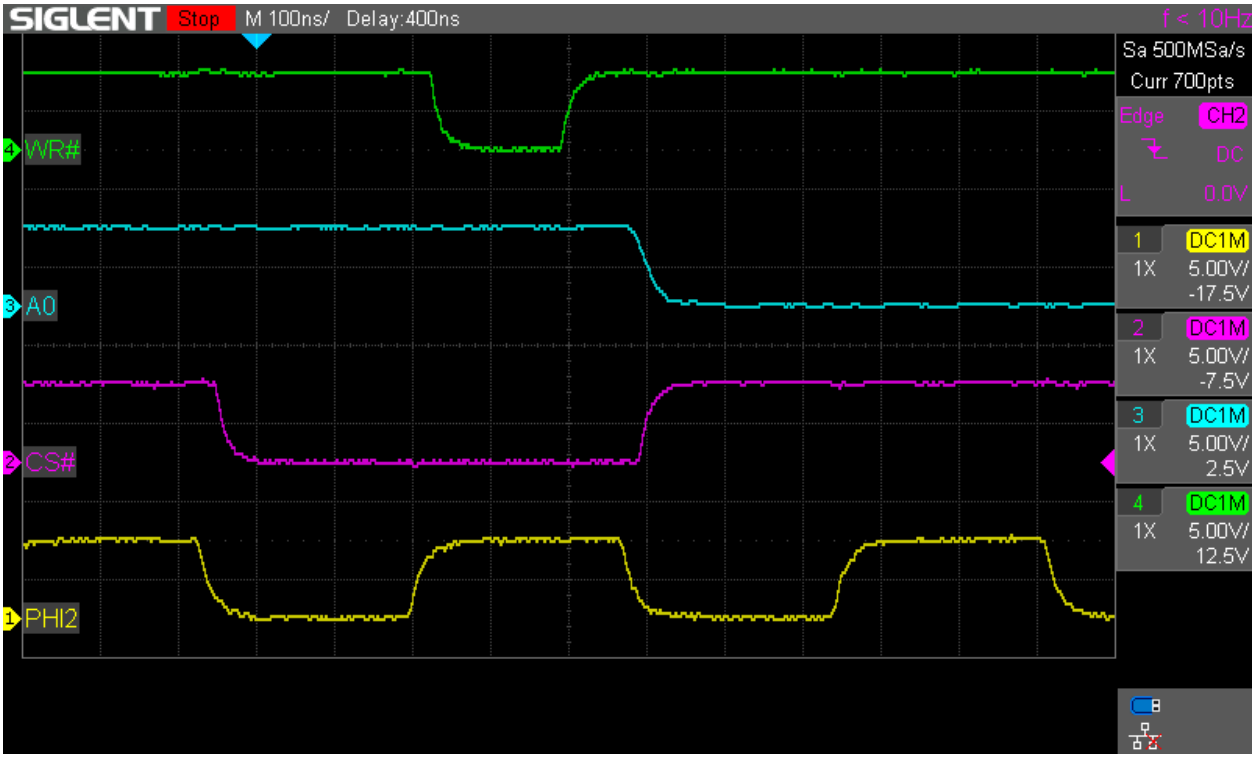


Figure 3: Write Operation Timing

### 3.2 Read Operation

Figure 4 shows the timing of a read operation. The $C/\overline{D}$ input is not shown here since the scope has but four analog channels. However, this signal follows the same timing as the $\overline{CS}$ signal. Here we can see that when $\overline{CS}$ is asserted, the $\overline{RD}$ signal goes low after a short delay. In response to the read request, the AM9511 pulls $\overline{PAUSE}$ low indicating that it is busy accessing the requested data. After the $\overline{PAUSE}$ signal rises, at the next rising edge of $PHI2$, the read operation is completed. By counting the rising edges between the transitions of $\overline{CS}$ from high to low and back to high again, we can see that a total of four cycles were needed to complete the read operation.

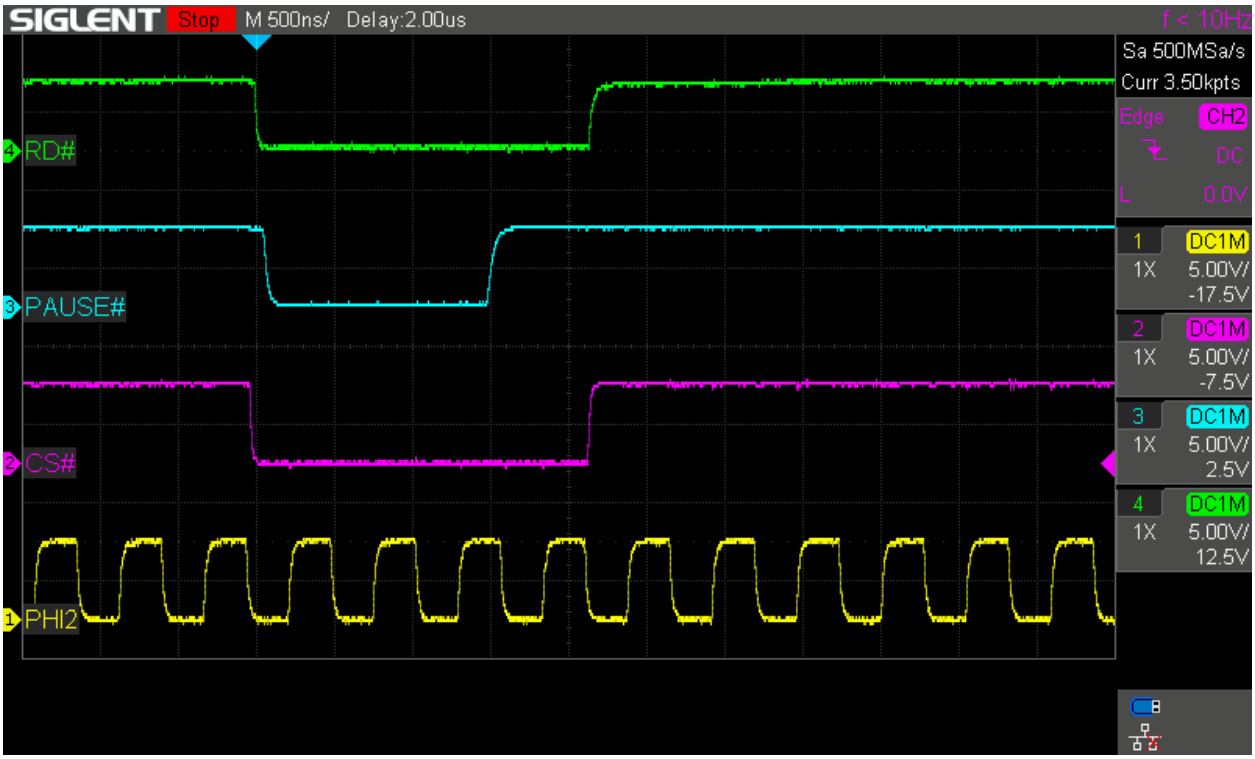


Figure 4: Read Operation Timing

### 3.3 Timing of the RDY Signal

In order to see the timing associated with the 6502's $RDY$ signal, in Figure 5, $\overline{RD}$ has been replaced with $RDY$. As shown in Figure 4, $\overline{RD}$ has essentially the same timing as $\overline{CS}$ so they can be considered equivalent for our purposes here.

Note that $RDY$ falls at virtually the same instant as $\overline{CS}$ as the first flip-flop is clocked to logic one by $\overline{CS}$ whenever the 6502's $R/\overline{W}$ signal is high. Subsequently, the $\overline{PAUSE}$ signal goes low, preventing the second flip-flop from clocking in logic one on subsequent falling edges of PHI2. Before the third falling edge of $PHI2$, $\overline{PAUSE}$ rises, and subsequently the second flip-flop clocks in a logic one, causing the first flip-flop to reset and asserting $RDY$. This allows the read operation to be completed at the next rising edge of $PHI2$.

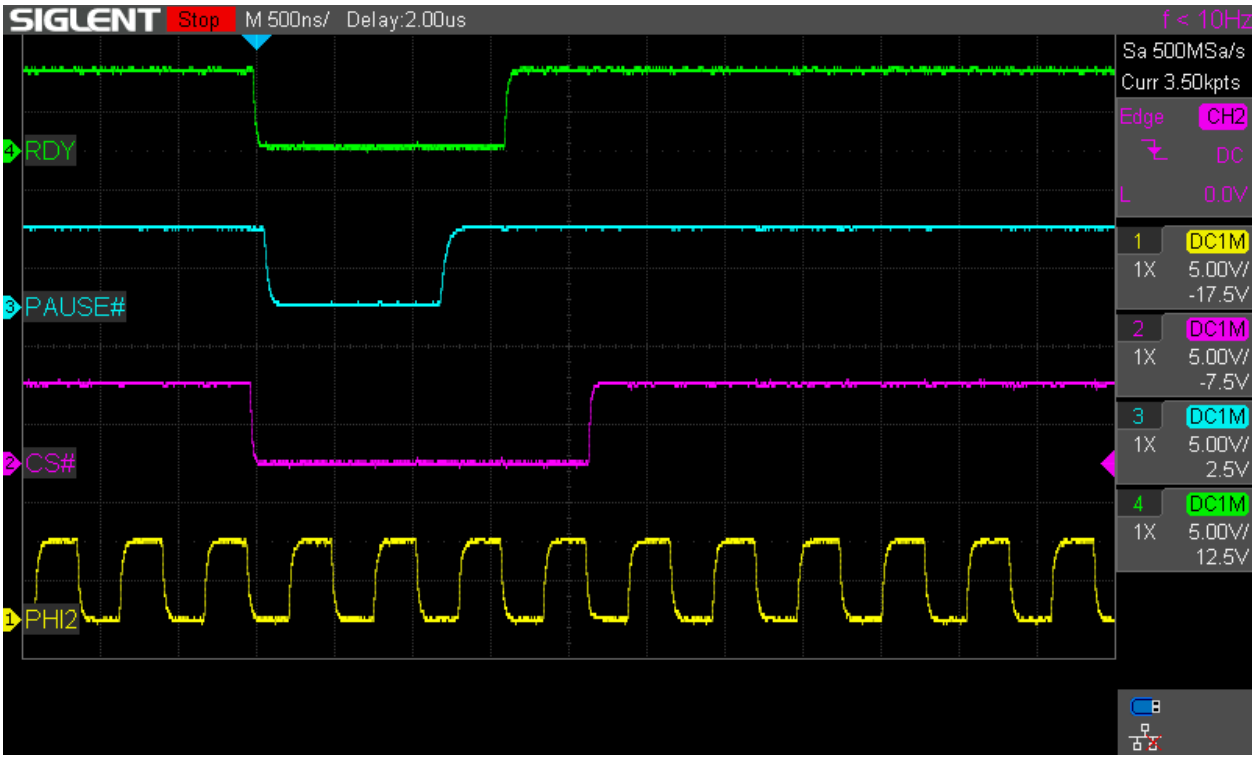


Figure 5: RDY Signal Timing